6
Claude Code Planning and Multi-agent Workflows

Agentic coding workflows benefit from structure, planning, and controlled execution. As systems grow more complex and incorporate more tools and automation, relying on ad hoc prompting increases the risk of unintended changes and unpredictable behavior.

This chapter examines how Claude Code supports structured agentic workflows through planning and multi-agent execution. It shows how moving from exploratory prompting toward spec-driven development improves safety and predictability, and how multiple agents can be coordinated to work on the same codebase efficiently. Although the examples use Claude Code, the principles discussed apply broadly to agentic systems.

NOTE

Claude Code planning mode is covered in the official documentation. For a concise overview of how planning mode works and when to use it, refer to the following:

`https://code.claude.com/docs/en/common-workflows#how-to-use-plan-mode`

In this chapter, we cover the following topics:

- Claude Code planning mode
- Using multiple Claude Code agents in parallel

Claude Code planning mode

Claude Code provides a planning mode that enables a transition from exploratory coding to spec-driven development. This mode is a foundational principle in agentic coding workflows. In planning mode, Claude Code is restricted to read-only operations and is responsible for producing a specification or implementation plan before any code changes are made.

Planning mode allows Claude Code to research, analyze, and reason about a problem space, then produce a detailed plan for implementation. Code changes are not permitted until the plan has been explicitly reviewed and approved.

Characteristics of planning mode

Planning mode is a read-only phase. Claude Code can read files, analyze the code base, make MCP calls, search documentation, and design solutions. It cannot edit, create, or delete files, make Git commits, install packages, or perform any execution-level actions.

This mode mirrors senior software engineering workflows. The process begins with gathering context and understanding requirements and business constraints. Planning occurs next, followed by execution only after review and approval.

Planning workflow

The planning workflow follows a structured sequence:

1. Claude Code researches and analyzes the existing codebase or the provided requirements.
2. A comprehensive implementation plan or specification is formulated
3. The plan is presented for review
4. Upon approval, Claude Code exits planning mode and proceeds with implementation.

This separation between strategy and execution provides consistency and safety, particularly when working with large or complex codebases.

By establishing a read-only planning phase, this workflow creates a structured foundation for making implementation decisions safely and deliberately. This foundation makes it possible to understand why planning artifacts, such as specifications, play a central role in more reliable development workflows, which are explored next.

Benefits of spec-driven development

Planning mode is especially useful when designing complex systems or understanding large projects. The primary benefit is the clear separation between planning and execution, which reduces risk and improves reliability.

A common practice is to export the generated plan or specification into a Markdown file. This file can be shared, reviewed collaboratively, and reused across the project lifecycle. This approach enhances the overall value of Claude Code by making planning artifacts explicit and persistent.

This workflow is an example of spec-driven development. Instead of immediately implementing features, time is spent defining the business problem, clarifying what is being solved, and determining how it will be solved. This approach leads to better outcomes and can reduce overall implementation time, particularly when combined with subagents.

A well-defined specification scopes the problem space for the language model, establishes clear boundaries, and reduces hallucinations. It prevents unintended side effects, such as unnecessary file changes or regressions in unrelated functionality.

The following example demonstrates how these principles are applied in practice through an end-to-end planning and refinement workflow.

Example: Iterating on the hook marketplace specification

In this exercise, we will use Claude Code's planning mode to draft a specification for a new Next.js project called HookHub. Instead of writing the implementation directly, we first define the structure and requirements of the project as a planning artifact. This approach allows us to iteratively refine the design before generating code.

> ```
> ### Hands-On: Using Planning Mode
>
> Open Claude Code in your project directory:
>
> claude
>
> Enter planning mode:
>
> /plan
> ```

The following example shows Claude Code running in planning mode after you enter the `/plan` command.

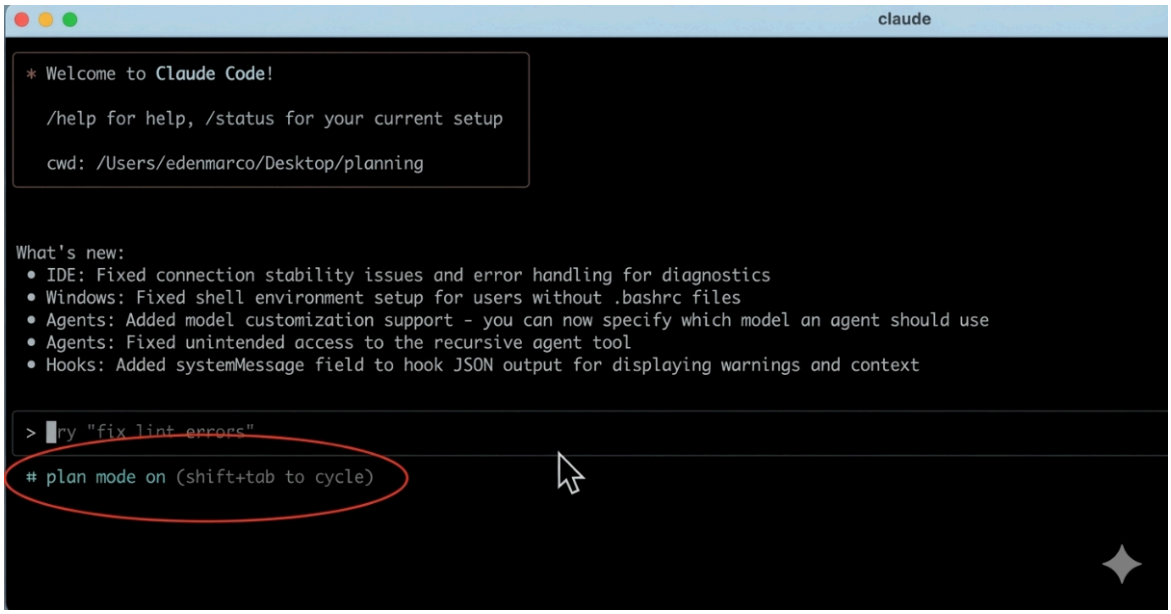

Figure 6.1 – Claude Code running in planning mode after executing the /plan command

While in planning mode, the following prompt is entered

Create a detailed specification for a web application called HookHub

The application should:

- *Display available Claude Code hooks*
- *Render them as cards*
- *Link each card to its GitHub repository*
- *Support filtering by hook event type*

Claude Code is instructed to generate a specification for a web application that serves as a marketplace for available Claude Code hooks. The application displays hooks as a collection of cards, with each card linking to a GitHub repository that contains a Claude Code hook intended to support developer workflows.

In planning mode, Claude Code generates an initial task list, gathers relevant information, and produces a specification document. During this phase, no code is written. Claude Code performs research, defines concepts, and outputs a structured specification intended for later implementation.

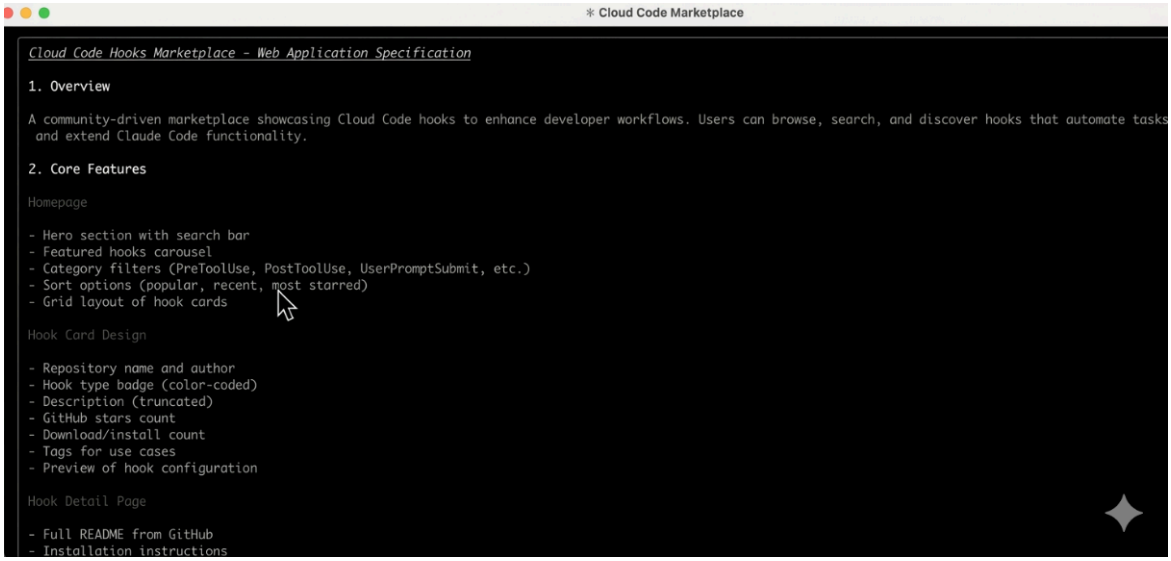

Figure 6.2 – Initial HookHub application specification generated in planning mode

The initial specification includes design and feature decisions that require refinement. In particular, hooks in this example use case are featured in a carousel layout, and only a subset of available hook event types is included.

The specification is refined through feedback. The carousel-based presentation is removed in favor of displaying all hook cards on the home page. Additional hook event types, including events triggered after code generation, are added to the specification.

Persisting the hook marketplace specification

After refinement is complete, the finalized specification for the hook marketplace application is exported to a Markdown file.

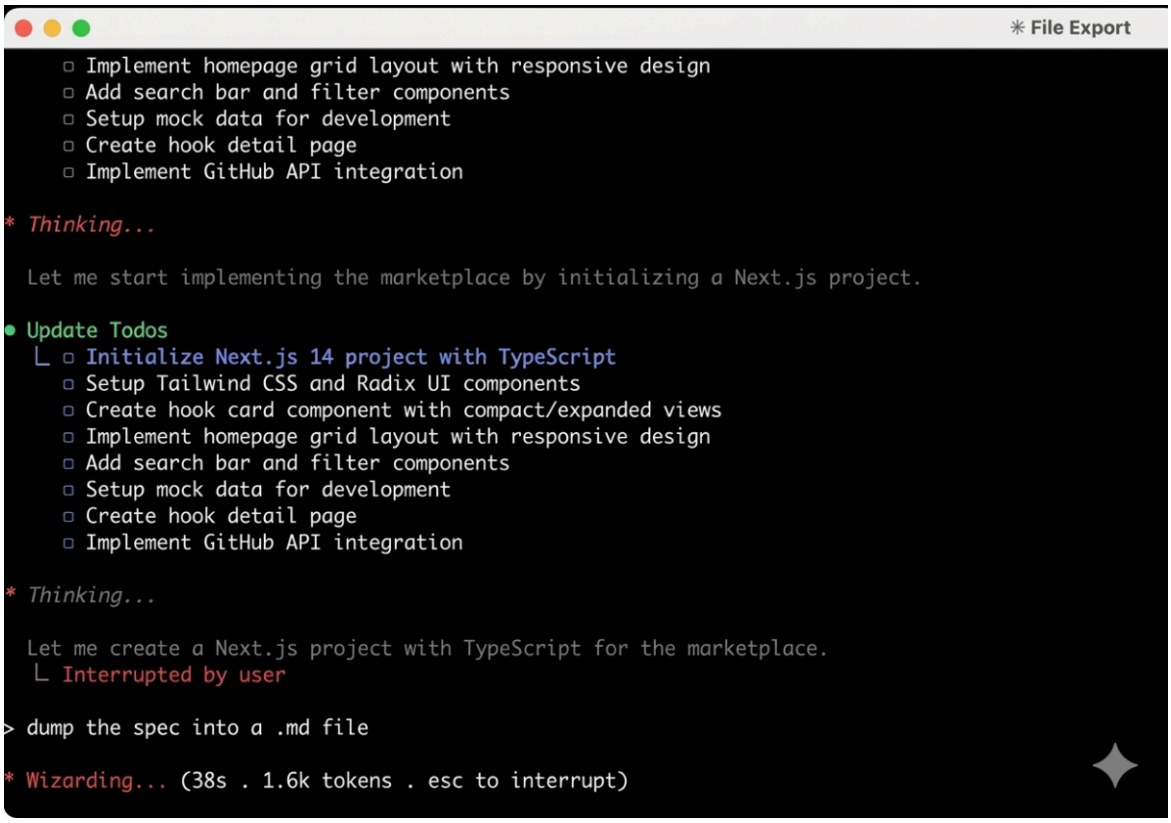

Figure 6.3 – Exporting and saving the HookHub specification as a Markdown file

The `.md` file can be committed to the repository or stored in a designated location, such as a `.claude` directory. The specification is retained as persistent project memory and reused as context for subsequent Claude Code requests.

This persisted specification constrains Claude Code's behavior during implementation and ensures consistent guidance when working with subagents.

This example showed how a concrete application specification is generated, refined, and persisted using planning mode and deep thinking. By iterating on real design decisions and retaining the resulting specification as project memory, the workflow establishes stable context for execution. With a specification in place, more advanced execution patterns, including parallel development, can be applied effectively.

Using multiple Claude Code agents in parallel

Claude Code supports running multiple AI agents simultaneously to accelerate development workflows. The core idea is to parallelize independent tasks so that more work can be completed in less time. This approach involves running multiple instances of Claude Code within the same project.

This workflow can be understood as an introductory example of multi-agent coding systems. While more complex agentic workflows involving sub-agents are possible, this setup demonstrates two independent Claude Code instances operating in parallel.

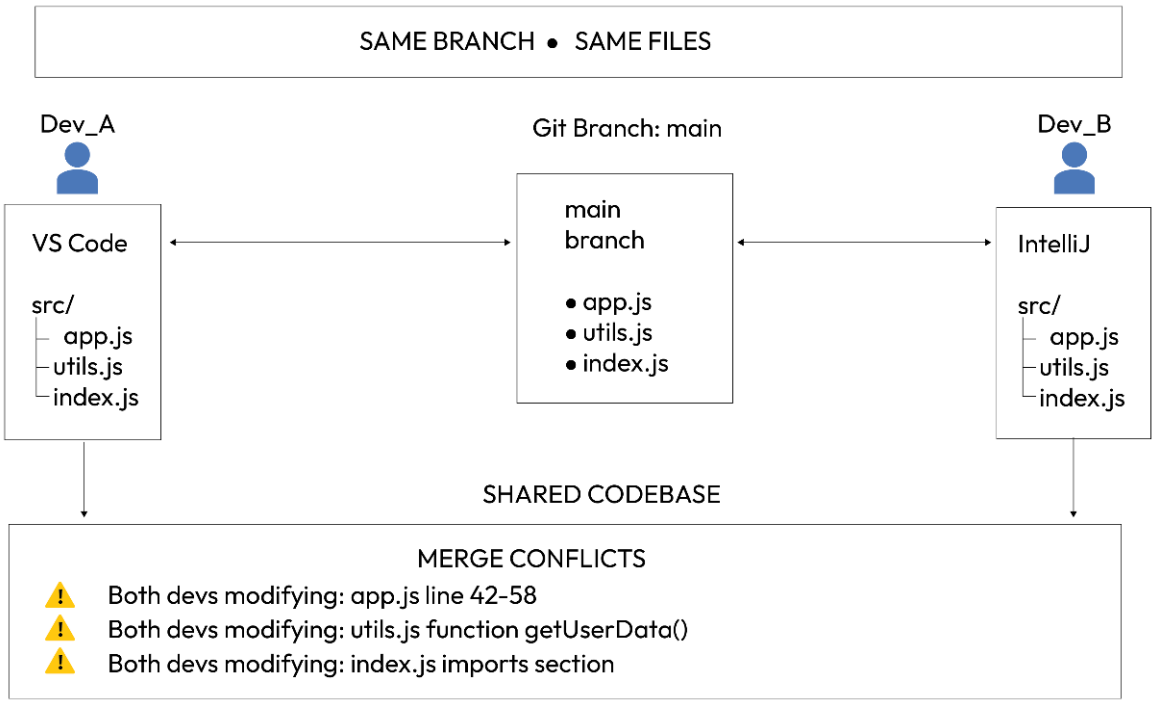

Figure 6.4 – Parallel Claude Code agents working on the same shared codebase

Multiple agents can be compared to multiple developers working on the same project and the same GitHub branch. Each agent has read and write access to the entire code base. This shared access enables collaboration but also requires careful consideration of task assignment to avoid conflicts. The effectiveness of this approach depends on the nature of the tasks assigned to each agent.

Identifying suitable tasks for parallel execution

Parallel execution is effective when tasks are independent. Examples of suitable tasks include fixing unrelated bugs, building separate UI pages, or creating independent components within a component library. These tasks can be executed concurrently without interfering with one another.

Tasks that depend on each other should not be executed in parallel. A common example is a full stack feature where a backend API endpoint is implemented and then consumed by a frontend. If separate agents work on the frontend and backend simultaneously, the frontend agent may attempt to consume an API that does not yet exist. This often results in mock implementations or assumptions that do not match the final API, leading to interface discrepancies.

A key skill in multi-agent workflows is identifying which tasks are truly independent and which are sequential. Independent tasks can be executed in parallel, while sequential tasks must be completed in order. In this workflow, the role of the engineer shifts from direct implementation to coordination and oversight of the work of AI agents.

Example: Parallel development in the HookHub project

This example shows how multiple Claude Code agents can work in parallel using the HookHub specification created earlier in the chapter. The specification defines the structure of the application and provides a shared reference for the implementation.

The project is based on the `project/ hookhub` branch, starting from a specific commit that contains the state of the application built previously. This commit represents the baseline from which the agents operate and defines the scope of the changes to be implemented.

```
### Establishing the Starting Point

This demo begins from the
 `project/
 hookhub
 `
 branch at commit:

<insert commit hash here>

To reproduce the starting state:

git checkout project/hookhub
git reset --hard <commit-hash>
npm install
npm run dev
```

At this point, the application displays:

- A basic hero section
- Hook cards rendered in a simple layout
- No visual refinements

The application displays hook cards and serves as the context for assigning parallel tasks to multiple agents.

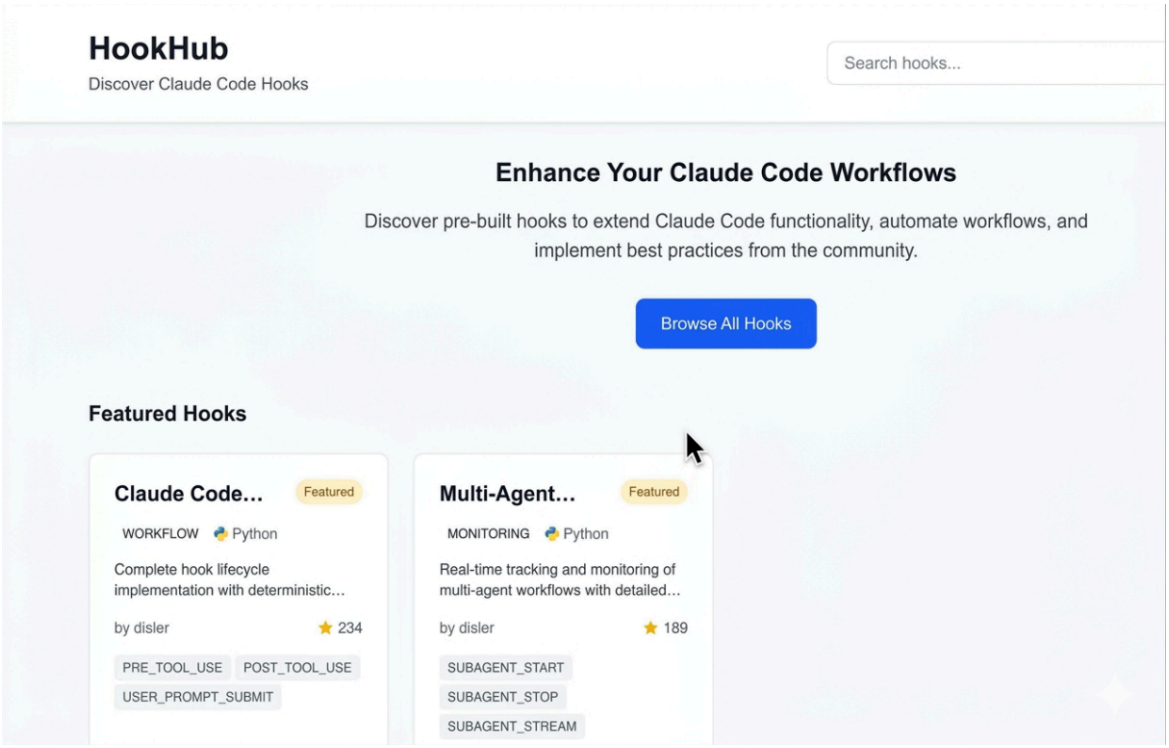

Figure 6.5 – HookHub interface used to identify independent tasks for parallel agent execution

To use multiple agents effectively, we first need to identify tasks that are truly independent. Independent tasks can be executed in parallel, which allows us to speed up development.

Assigning independent tasks to agents

In the current HookHub interface, two parts of the page can be improved separately, making them good candidates for parallel work. Two independent tasks are identified:

- Updating the hook card component to improve its visual design
- Updating the hero section of the main page

Figure 6.6 – Two Claude Code instances running in parallel to update different parts of the HookHub interface

Each task is assigned to a separate Claude Code instance. One agent is instructed to modify the hook card component file, while the other agent is instructed to modify the hero section.

```
### Running Multiple Claude Code Instances

Open two separate terminal windows.

Terminal 1 – Update Hook Card Component

claude
```

```
Prompt:
Improve the visual design of the HookCard component.
Modify only:
src/components/HookCard.tsx
Do not modify other files.

Terminal 2 – Update Hero Section

claude

Prompt:

Redesign the hero section.
Modify only:
src/components/Hero.tsx
Do not modify HookCard.tsx.
```

Both agents now operate independently in parallel.

This setup works effectively because the hook card component and the hero section are implemented in separate files. If both components were defined in the same file, concurrent modifications would introduce conflicts.

Separating components into individual files improves maintainability, readability, and testability. This is a general best practice in software development and is especially important when using agentic workflows.

Executing tasks concurrently

Both Claude Code instances operate independently and execute their assigned tasks in parallel. Each agent modifies only the files relevant to its task. Because the components are isolated, no file conflicts occur during execution.

After execution completes, the updated hook card component and the updated hero section reflect their assigned changes.

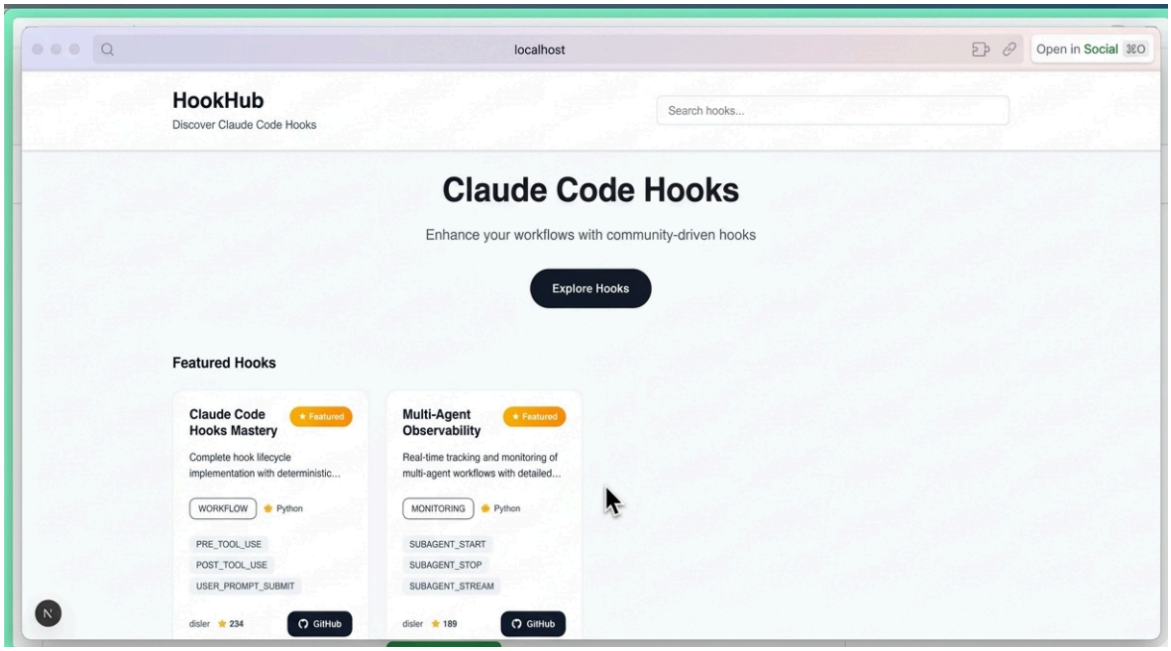

Figure 6.7 – HookHub interface after concurrent updates to the hook cards and the hero section

Reviewing and committing changes

Once both agents complete their work, the changes are reviewed and committed to the repository. The commit captures the improvements made to the hook card component and the hero section as a result of the multi-agent workflow.

After both agents complete their work, review changes:

```
git status
git diff
```

If the changes look correct:

```
git add .
git commit -m "Refine HookCard and Hero using parallel Claude Code agents"
```

This example demonstrates how multiple Claude Code instances can be used concurrently to accelerate development when tasks are independent and properly scoped.

NOTE

The example in this chapter intentionally demonstrates the simplest form of multi-agent execution: multiple Claude Code instances operating on the same branch with shared file access, modifying separate files to avoid conflicts.

This setup is useful for understanding the core idea of parallel execution. However, running multiple agents on the same branch is inherently fragile. If agents modify overlapping files or introduce incompatible changes, conflicts can occur.

In professional development environments, the industry-standard approach

own branch, with an independent copy of the codebase. Changes are later reviewed and merged in a controlled manner.

This chapter focuses on conceptual clarity and the simplest reproducible setup. More robust parallelization strategies, including branch isolation and structured merge workflows, are covered later in this book.

Summary

In this chapter, we explored how Claude Code supports agentic development workflows through planning mode and multi-agent execution. We examined how planning mode enables spec-driven development by separating analysis from implementation, and how specifications can be refined using deep thinking and persisted

as project memory. We also demonstrated how multiple Claude Code agents can work in parallel on independent tasks within a shared codebase.

In the next chapter, we'll explore Claude Code subagents and how they enable structured, isolated, and concurrent workflows within complex development tasks.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`), search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website do not require an invoice.